

Reviewer Pack — Kahler Manifolds and Resolution Tree Width

Entry ce78108b09a3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 16:01:29 UTC

Kahler Manifolds and Resolution Tree Width

Entry ID: ce78108b09a3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 16:01:29 UTC

1. Conjecture

Field A (mathematical branch): Kähler geometry **Field B** (complexity object): resolution tree width

Statement:

For any given 3-CNF formula φ with n variables, the resolution tree width $W(\varphi)$ is upper-bounded by a constant multiple of the Kähler potential of the associated Kähler manifold $X(\varphi)$. Specifically, $W(\varphi) \leq c \cdot K_X(\varphi)$, where $K_X(\varphi)$ is the Kähler potential evaluated at the point representing φ in $X(\varphi)$.

Rationale (proposer's reasoning):

The structure of a Kähler manifold encodes rich geometric information, which may reveal hidden patterns and dependencies within the logical structure of a 3-CNF formula. Exploring such a connection could potentially provide insights into the inherent complexity of resolution proofs.

Taxonomy category: Kahler_geometry Resolution_tree_width (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 34d1561ac1158551

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all 3-CNF formulas with $n \leq 40$ and 30 random seeds, the ratio of resolution tree width $W(\varphi)$ to Kähler potential $K_X(\varphi)$, calculated as $c = W(\varphi) / K_X(\varphi)$, remains below a pre-defined threshold of 5 for at least 80% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Kähler geometry" AND "resolution tree width" AND CNF - "upper bound" Kähler potential resolution tree width CNF - CNF formula Kähler manifold Kähler geometry resolution tree width

Top relevant hits considered: - [http://arxiv.org/abs/1109.2952v4] Upper bound on distance in the pants complex - [http://arxiv.org/abs/1803.07376v1] Sub-exponential Upper Bound for #XSAT of some CNF Classes - [http://arxiv.org/abs/hep-ph/0612133v4] The upper bound on the lightest Higgs mass in the NMSSM revisited - [http://arxiv.org/abs/1510.00510v3] Okounkov bodies and the Kähler geometry of projective manifolds - [http://arxiv.org/abs/1209.3373v2] On the structure of co-Kähler manifolds - [http://arxiv.org/abs/1711.09881v2] Coupled Kähler-Ricci solitons on toric Fano manifolds

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(x == 0 for x in clause):
                clause[random.randint(0, n - 1)] = random.choice([-1, 1])
            clauses.append(clause)
        return clauses

    def resolution_tree_width(clauses):
        # Simplified version of resolution tree width calculation
        return len(clauses)

    def kahler_potential(n):
        # Simplified version of Kähler potential calculation
        return n * math.log(n)

    n = random.randint(5, 40)
    clauses = generate_3cnf(n)
    w_phi = resolution_tree_width(clauses)
    k_x_phi = kahler_potential(n)
```

```

if k_x_phi == 0:
    return {
        "metric_name": "c",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "Kähler potential is zero"
    }

c = w_phi / k_x_phi
return {
    "metric_name": "c",
    "metric_value": c,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": c <= 5,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_c = sum(r['metric_value'] for r in results) / len(results)
    std_c = math.sqrt(sum((r['metric_value'] - mean_c) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_c} std={std_c} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_c} std={std_c} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
        print(f"RESULT: FALSIFIED counterexample=\"c > 5\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

421, "metric_name": "c", "metric_value": 0.6569174775061022, "instances_tested": 1, "n_max": 21, "conjecture_holds": True, "counterexample": ""
TRIAL: {"seed": 463, "metric_name": "c", "metric_value": 0.6919525125223871, "instances_tested": 1, "n_max": 21, "conjecture_holds": True, "counterexample": ""}
TRIAL: {"seed": 503, "metric_name": "c", "metric_value": 0.7385387461377101, "instances_tested": 1, "n_max": 21, "conjecture_holds": True, "counterexample": ""}
TRIAL: {"seed": 547, "metric_name": "c", "metric_value": 1.0277966847395015, "instances_tested": 1, "n_max": 21, "conjecture_holds": True, "counterexample": ""}
TRIAL: {"seed": 593, "metric_name": "c", "metric_value": 0.6213349345596119, "instances_tested": 1, "n_max": 21, "conjecture_holds": True, "counterexample": ""}

```

```

TRIAL: {"seed": 631, "metric_name": "c", "metric_value": 0.6378579778076021, "instances_tested": 1, "
TRIAL: {"seed": 677, "metric_name": "c", "metric_value": 0.9102392266268374, "instances_tested": 1, "
TRIAL: {"seed": 727, "metric_name": "c", "metric_value": 0.5498151554849872, "instances_tested": 1, "
TRIAL: {"seed": 773, "metric_name": "c", "metric_value": 0.7213475204444817, "instances_tested": 1, "
TRIAL: {"seed": 821, "metric_name": "c", "metric_value": 0.5581106265512473, "instances_tested": 1, "
TRIAL: {"seed": 877, "metric_name": "c", "metric_value": 0.6792465437902172, "instances_tested": 1, "
TRIAL: {"seed": 929, "metric_name": "c", "metric_value": 0.7059122477295223, "instances_tested": 1, "
RESULT: SUPPORTED mean=0.706077142857425 std=0.13493243675614672 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code calculates the resolution tree width as the number of clauses, which is a trivial metric for 3-CNF formulas and does not capture the complexity of the resolution process. Additionally, the Kähler potential calculation is simplified and does not correspond to the actual mathematical definition of the Kähler potential for Kahler manifolds.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code calculates the resolution tree width as the number of clauses, which is a trivial metric and does not capture the complexity of the resolution process. The Kähler potential calculation is also simplified and does not correspond to the actual mathematical definition. The critic's challenge indicates that the pre-registered support condition may not be unambiguously met. | next: Re-evaluate the conjecture with a more accurate method for calculating the resolution tree width and the K

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	29465
2	preregistration	ollama_remote	glm4:latest	0	0	14306
3	novelty	ollama_remote	glm4:latest	0	0	19470
4	novelty	ollama_remote	glm4:latest	0	0	12659
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18825
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16761
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15056
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9748
9	critic	ollama_remote	glm4:latest	0	0	11373
10	judge	ollama_remote	glm4:latest	0	0	12065

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 159727 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/ce78108b09a3.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ce78108b09a3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ce78108b09a3.tar.gz` (if generated)